
Terraform Building Blocks

Terraform is built on **seven core components**. Understanding these is essential before writing any real Terraform code.

1 PROVIDERS

Providers are the **plugins** Terraform uses to interact with cloud platforms, SaaS tools, or infrastructure systems.

✓ What Providers Do:

- Authenticate to platforms (AWS, Azure, GCP, etc.)
- Define available resource types
- Communicate with APIs
- Manage resource lifecycle

✓ Examples of Providers:

- `aws`
- `azurerm`
- `google`

- [kubernetes](#)
- [cloudflare](#)
- [github](#)
- [vault](#)
- [helm](#)

◆ **Example: AWS Provider**

```
provider "aws" {  
  region = "ap-south-1"  
}
```

◆ **Example: Multiple Provider Instances**

```
provider "aws" {  
  region = "us-east-1"  
}  
  
provider "aws" {  
  alias  = "mumbai"  
  region = "ap-south-1"  
}
```

◆ **Example: Provider Versioning**

```
terraform {
```

```
required_providers {  
  aws = {  
    source = "hashicorp/aws"  
    version = "~> 5.0"  
  }  
}
```

2 RESOURCES

A Resource is the **actual infrastructure component** Terraform creates or manages.

✓ Examples of Resources:

- AWS EC2 → `aws_instance`
- AWS S3 → `aws_s3_bucket`
- Azure VM → `azurerm_linux_virtual_machine`
- GCP Compute Instance → `google_compute_instance`
- Kubernetes Deployment → `kubernetes_deployment`

Resources always follow this structure:

```
resource "<PROVIDER>_<RESOURCE_TYPE>" "<NAME>" {  
  # configuration arguments  
}
```

♦ Example: EC2 Instance

```
resource "aws_instance" "my_server" {  
  ami      = "ami-0abcd12345"  
  instance_type = "t2.micro"  
}
```

◆ Example: S3 Bucket

```
resource "aws_s3_bucket" "logs" {  
  bucket = "my-log-bucket"  
}
```

◆ Example: Security Group

```
resource "aws_security_group" "web" {  
  name      = "web-sg"  
  description = "Allow HTTP traffic"  
  
  ingress {  
    from_port = 80  
    to_port   = 80  
    protocol  = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```

③ VARIABLES (Input Variables)

Variables make your Terraform code **dynamic**, **flexible**, and **reusable**.

✓ Why Variables Are Used:

- Avoid hardcoding
 - Pass environment-specific values
 - Use same code for dev/stage/prod
 - Reduce duplication
-

♦ **Example: Declare a Variable**

```
variable "instance_type" {  
  type      = string  
  default   = "t2.micro"  
  description = "EC2 instance size"  
}
```

♦ **Example: Use the Variable**

```
resource "aws_instance" "web" {  
  instance_type = var.instance_type  
}
```

♦ **Passing Variables Through .tfvars**

Create: `terraform.tfvars`

```
instance_type = "t3.medium"
```

Run:

terraform apply

4 OUTPUTS

Outputs allow you to **display useful information** after Terraform finishes deploying resources.

✓ Common Use Cases:

- Print instance public IP
 - Show database endpoint
 - Pass values to modules
 - Output credentials to CI/CD
-

♦ Example: Output Public IP

```
output "public_ip" {  
  value = aws_instance.web.public_ip  
}
```

♦ Example: Output VPC ID

```
output "vpc_id" {  
  value = aws_vpc.main.id  
}
```

5 MODULES

Modules allow you to **group Terraform code** and **reuse it** across environments or teams.

You can create your own modules OR use official registry modules.

✓ Why Modules Are Useful:

- Reusability
- Clean code
- Team standardization
- DRY (Don't Repeat Yourself)

♦ Example: Using a Module

```
module "vpc" {  
  source = "./modules/vpc"  
  cidr   = "10.0.0.0/16"  
}
```

♦ Example: Using a Public Registry Module

```
module "eks" {  
  source = "terraform-aws-modules/eks/aws"  
  version = "19.0.0"  
  
  cluster_name = "prod-eks"  
  subnets     = var.subnets
```

```
}
```

◆ **Module Structure**

```
modules/  
  vpc/  
    main.tf  
    variables.tf  
    outputs.tf
```

⑥ **DATA SOURCES**

Data sources allow Terraform to **read existing information** instead of creating new resources.

✓ **Common Use Cases:**

- Get latest AMI for EC2
- Read existing VPC/Subnet
- Fetch route53 DNS records
- Get SSM parameter values
- Pull secrets from Vault
- Fetch GitHub repo information

Data source block:


```
data "<PROVIDER>_<TYPE>" "<NAME>" {  
  # query arguments  
}
```

◆ Example: Get Latest AMI

```
data "aws_ami" "latest_amazon_linux" {  
  most_recent = true  
  owners      = ["amazon"]  
  
  filter {  
    name   = "name"  
    values = ["amzn2-ami-hvm-*"]  
  }  
}
```

Use it:

```
ami = data.aws_ami.latest_amazon_linux.id
```

◆ Example: Read Existing VPC

```
data "aws_vpc" "default" {  
  default = true  
}
```

7 LOCAL VALUES

Local values are like **variables inside your Terraform configuration**, used to simplify expressions or avoid repetition.

They do NOT accept input — they are defined and used internally.

♦ **Example: Local Tags**

```
locals {
  common_tags = {
    Environment = "dev"
    Owner       = "team-a"
  }
}
```

Use:

```
resource "aws_instance" "web" {
  tags = local.common_tags
}
```

♦ **Example: Combine Strings**

```
locals {
  full_name = "${var.env}-${var.app_name}"
}
```

 Summary Table — Terraform Building Blocks

Block	Purpose	Example
Providers	Connect to cloud/services	<code>provider "aws"</code>

Resources	Create infra	<code>aws_instance,</code> <code>aws_s3_bucket</code>
Variables	Accept inputs	<code>variable "instance_type"</code>
Outputs	Show values	<code>output "public_ip"</code>
Modules	Reusable code	<code>module "vpc"</code>
Data Sources	Read existing infra	<code>data "aws_ami" ...</code>
Local Values	Internal reusable variables	<code>locals { common_tags = ... }</code>
